南科大环境学院
SUSTech | School of Environment

Computing and Programming for Environmental Research



Name	孙铠璐
Student ID	12632737
Major	Environmental Science and Technology
Class Date	09/30
Assignment	Assignment 1

1、Flowchat

Function:taking (0,1) decimals as examples

[2]:
```
#Flowchat
import random
def print_values():
    a=random.random()
    b=random.random()
    c=random.random()
    print("a:",a,"b:",b,"c:",c)
    if a>b:
        if b>c:
            print(a,b,c)
        elif b<=c and a>c:
            print(a,c,b)
        elif b<=c and a <= c:
            print(c,a,b)
    #当a>b为false时
    else:
        #该条件下无输出结果，即a<=b且b>c时
        if b>c:
            return
        elif b<=c:
            print(c,b,a)

print_values()
```

Result:

```
a: 0.3095543867362649 b: 0.25832110640170425 c: 0.8636071556955482
0.8636071556955482 0.3095543867362649 0.25832110640170425
```

While a<=b and b>c:

```
a: 0.3249956177218667 b: 0.7704147661360076 c: 0.6809312786582694
```

2、Matrix multiplication

Function:

[6]:
```
#Matrix multiplication
import random
import numpy as np

M1=np.random.randint(0,51,(5,10))
M2=np.random.randint(0,51, (10,5))

print("The matrix M1 is:\n", M1)
print("The matrix M2 is:\n", M2)

def Matrix_Multip(M1,M2):
    result=np.zeros((len(M1),len(M2[0])),dtype=int)
    for i in range(len(M1)):
        for j in range(len(M2[0])):
            for k in range(len(M2)):
                result[i][j]+=M1[i][k]*M2[k][j]
    print("The result of the matrix multiplication is:\n", result)

Matrix_Multip(M1,M2)
```

Result:

```
The matrix M1 is:
 [[ 2  2  1 20 19 30 21  5 10 42]
 [21 43 29 10 42 46 25 36 12  7]
 [31 26 19 28 31 30  8  6 11 26]
 [39 37 36  2  0 30 32 45 28 45]
 [14 43 28 39 37 11  1 44  4 11]]
The matrix M2 is:
 [[20 14 25 20 37]
 [31 36 34 16 15]
 [49 42  7  8 36]
 [42 30  9 45 41]
 [10  3  1 31 26]
 [28 46  6 24 24]
 [10 14 48 42 46]
 [36 48 20 30 16]
 [ 0 20 31 21 21]
 [ 4 18 46 18 45]]
The result of the matrix multiplication is:
 [[2579 3669 3854 4287 5320]
 [6876 8046 5212 6704 7365]
 [5083 5569 4296 5344 6760]
 [6735 8808 8057 6562 8901]
 [6939 7111 4020 6002 6325]]
```

3、Pascal triangle

Function:

```python
#Pascal triangle
def Pascal_triangle(k):
    triangle = []
    for i in range(k):
        row = [1] * (i + 1)
        for j in range(1, i):
            row[j] = triangle[i - 1][j - 1] + triangle[i - 1][j]
        triangle.append(row)
    return row

print(Pascal_triangle(100))
print(Pascal_triangle(200))
```

Result:Pascal_triangle(100)

```
[1, 99, 4851, 156849, 3764376, 71523144, 1120529256, 14887031544, 171200862756, 1731030945644, 15579278510796, 126050526132804, 924370524973896, 6186171974825304, 38000770702498296, 215337700647490344, 1130522928399324306, 5519611944537877494, 25144898858450330806, 107196674080761936594, 428786696323047746376, 1613054714739084379224, 5719012170438571889976, 19146258135816088501224, 60629817430084280253876, 181889452290252840761628, 517685364210719623706172, 1399667836569723427057428, 3599145865465003098147672, 8811701946483283447189128, 20560637875127661376774632, 45764000431735762419272568, 95724850091743849514095420 7, 197443926105102399225573693, 383273503615787010261407757, 711793649572175876199757263, 1265410932572757113244012912, 2154618614921181030658724688, 3379665916777489877819632612 8, 5044567227278209667406248628, 7139364957217587619975726 3, 9674339023040098565708730672, 12765181204636128081852426683 2, 15746189748319168690076042183 2, 18686997258882811497847533 68, 21726423750712434928840495368, 24477404874816869007604218 32, 27651812046361280818524266832, 30796659167774898778196326128, 33796659167774898778196326128, 36743390230400985657087306 72, 39674339023040098565708730672, 42473914826002394093579920 6928, 44967410615025936013782474172, 47139148260023940935799206928, 48467410615025936013782474172, 48467410615025936013782474172, 47473914826002394093579920 6928, 44967410615025936013782474172, 42473914826002394093579920 6928, 39674339023040098565708730672, 36743390230400985657087306 72, 33796659167774898778196326128, 30796659167774898778196326128, 27651812046361280818524266832, 24477404874816869007604218 32, 21726423750712434928840495368, 19186899725888281149874753368, 16390109145274293016493707032, 13796659167774898778196326128, 11686899725888281149874753368, 9724850091743849514095420 7, 7936465916777489877819632612 8, 6321726423750712434928840495368, 4876541061502593601378247417 2, 3674339023040098565708730672, 2706481204636128081852426683 2, 1961890452290252840761628, 1399667836569723427057428, 9674339023040098565708730672, 6629817430084280253876, 4287866963230477463 76, 2701966740807619365 94, 1613054714739084379224, 9119012170438571889976, 5176853642107196237061 72, 2856063787512766137677463 2, 1471200862756, 14887031544, 1120529256, 71523144, 3764376, 156849, 4851, 99, 1]
```

Result:Pascal_triangle(200)

[1, 199, 19701, 1293699, 63391251, 2472258789, 79936367511, 2203959847089, 52895036330136, 1122550215450664, 2132845
4093562616, 366461620334848584, 5741232051912627816, 82585414900589338584, 1097206226536401212616, 13532210127282281
622264, 1556204164637462386560 36, 167520801252150362788 5564, 16938214348828536681954036, 161358778796735007338614766
4, 145222900917061506604753287 6, 123785234591209569915480183 24, 100153507987433197477070330076, 77074656146850765014
9628192324, 565214145076905610109727341037 6, 395649901553833927076809138726 32, 264781087962950397351403038993768, 16
9656030435520069414047132392303 2, 104217275838962328354343238469557 68, 614522557533191660296299785458426 32, 34822944
926880860750123654509310824 8, 189841215891705337637770890712049335 2, 996666383431453022598297176238259009 8, 50437359
403955349931489584373269471102, 246252990031076120253743264881256829498, 116090695300364456691050396301163933906 2, 5
288576119238825249258962498164134766838, 232983218225926625845732672216419991079 62, 993244246121055615447597181554215
154091838, 410031599039717830992469605718533482276562, 164012639615887132396987842287413392910624 8, 6360490170469769
280761235835048470603119352, 239275582603386558657208395699442465545918 48, 873634103923992783934458561042150397458353
352, 309743000482142896122217126187671504553416248, 106689255721626997553208121242420184900172263 2, 3571770735028382
091998706667681023581492775768, 116272536693477119165064280884084384674126530 32, 368196366196010877356036889466267218134734012 68, 11627253669
347711916506428088404384674126530 32, 357177073502838209199870666768102358149277576 8, 106689255721626997553208121242
4201849017322632, 309743000482142896122217126187671504553416248, 873634103923992783934458561042150397458353 52, 23927
558260338655865720839569944246554591848, 636049017046976928076123583504847060311935 2, 164012639615887132396987842287
4133929106248, 410031599039717830992469605718533482276562, 993244246121055615447597181554211540918 38, 23298321822592
662584573267221641999107962, 528857611923882524925896249816413476683 8, 116090695300364456691050396301163933906 2, 246
252990031076120253743264881256829498, 504373594039553499314895843732694711 02, 996666383431453022598297176238259009 8,
189841215891705337637770890712049335 2, 348229449268808607501236545093108248, 614522557533191660296299785458426 32, 10
421727583896232835434323846955768, 169656030435520069414047132392303 2, 264781087962950397351403038993768, 3956499015
538339270768091387263 2, 565214145076905610109727341037 6, 770746561468507650149628192324, 100153507987433197477070330
076, 123785234591209569915480183 24, 145222900917061506604753287 6, 161358778796735007338614764, 169382143488285366819
54036, 167520801252150362788556 4, 155620416463746238656036, 135322101272822816222 64, 109720622653640121261 6, 8258541
4900589338584, 5741232051912627816, 366461620334848584, 21328454093562616, 1122550215450664, 52895036330136, 2203959
847089, 79936367511, 2472258789, 63391251, 1293699, 19701, 199, 1]

4、Add or double

Function:

```
#Add or double
def Least_moves():
    import random
    RMB=1
    step=0
    x=random.randint(1,101)
    print("The random number x is:", x)

    while x>1:
        if x%2==0:
            x=x//2
        else:
            x=x-1
        step+=1
    print("The minimum number of steps is:", step)

Least_moves()
```

Result：

```
The random number x is: 37
The minimum number of steps is: 7

The random number x is: 5
The minimum number of steps is: 3
```

5、Dynamic programming

5.1：

```
#Dynamic programming
#5.1
#我从https://www.cnblogs.com/timjames/p/8657312.html获取了思路
def Find_expression():
    import random
    num="123456789"
    result=[]
    target=random.randint(1,101)

    options=["+","-",""]

    for i in range(3):
        for j in range(3):
            for k in range(3):
                for l in range(3):
                    for m in range(3):
                        for n in range(3):
                            for o in range(3):
                                for p in range(3):
                                    expression=num[0]+options[i]+num[1]+options[j]+num[2]+options[k]+num[3]+options[
                                    if eval(expression)==target:
                                        result.append(expression)


    print("The expressions that equal", target, "are:", result)

Find_expression()
```

Result：

```
The expressions that equal 46 are: ['1+2+34+5-6-7+8+9', '1+2+34-5+6+7-8+9', '1+2-3-4+56-7-8+9', '1+23-4-56-7+89', '1
-2+3+4-56+7+89', '1-2+3-4+56-7+8-9', '1-2+34-5-6+7+8+9', '1-2+34-56+78-9', '1-2-3+4+56+7-8-9', '1-2-34+5-6-7+89', '1
-23+4+56+7-8+9', '12+3+4+5-67+89', '12+3+45-6-7+8-9', '12+3-4+5+6+7+8+9', '12+3-45-6-7+89', '12-34+5-6+78-9', '123+4
-5+6+7-89', '123-4-5-67+8-9']

The expressions that equal 76 are: ['1+2+3+4+56-7+8+9', '1+2-3-4+5+6+78-9', '1+2-3-4+56+7+8+9', '1+2-34+5+6+7+89',
'1+23+4+56-7+8-9', '1-2+3+4-5+6+78-9', '1-2-3+4-5-6+78+9', '12+3+45+6-7+8+9', '12-3+4-5+67-8+9', '12-3+45-67+89', '1
2-3-4+5+67+8-9', '12-3-4-5-6-7+89', '12-34+5+6+78+9', '123-45+6-7+8-9']
```

5.2：

```python
#5.2
def Find_expression():
    num="123456789"
    import numpy as np
    options=["+","-",""]
    result_num=np.zeros((100,2),dtype=int)

    for target in range(1, 101):
        count = 0
        for i in range(3):
            for j in range(3):
                for k in range(3):
                    for l in range(3):
                        for m in range(3):
                            for n in range(3):
                                for o in range(3):
                                    for p in range(3):
                                        expression=(num[0]+options[i]+num[1]+options[j]+num[2]+options[k]+num[3]+opt
                                        if eval(expression) == target:
                                            count += 1
        result_num[target-1,0] = target
        result_num[target-1,1] = count
    print(result_num)

    max_count = max(result_num[:,1])
    min_count = min(result_num[:,1])

    max_i = result_num[np.where(result_num[:,1] == max_count), 0]
    min_i = result_num[np.where(result_num[:,1] == min_count), 0]

    print("The maximum number of expressions is:", max_count, "for i:", max_i)
    print("The minimum number of expressions is:", min_count, "for i:", min_i)


Find_expression()
```

Result(part)：

```
 [ 80   9]
 [ 81  19]
 [ 82  12]
 [ 83  13]
 [ 84  13]
 [ 85  12]
 [ 86  11]
 [ 87  12]
 [ 88   6]
 [ 89  12]
 [ 90  14]
 [ 91  16]
 [ 92  13]
 [ 93  11]
 [ 94  11]
 [ 95  10]
 [ 96  11]
 [ 97   7]
 [ 98   9]
 [ 99  17]
 [100  11]]
The maximum number of expressions is: 26 for i: [[ 1 45]]
The minimum number of expressions is: 6 for i: [[88]]
```